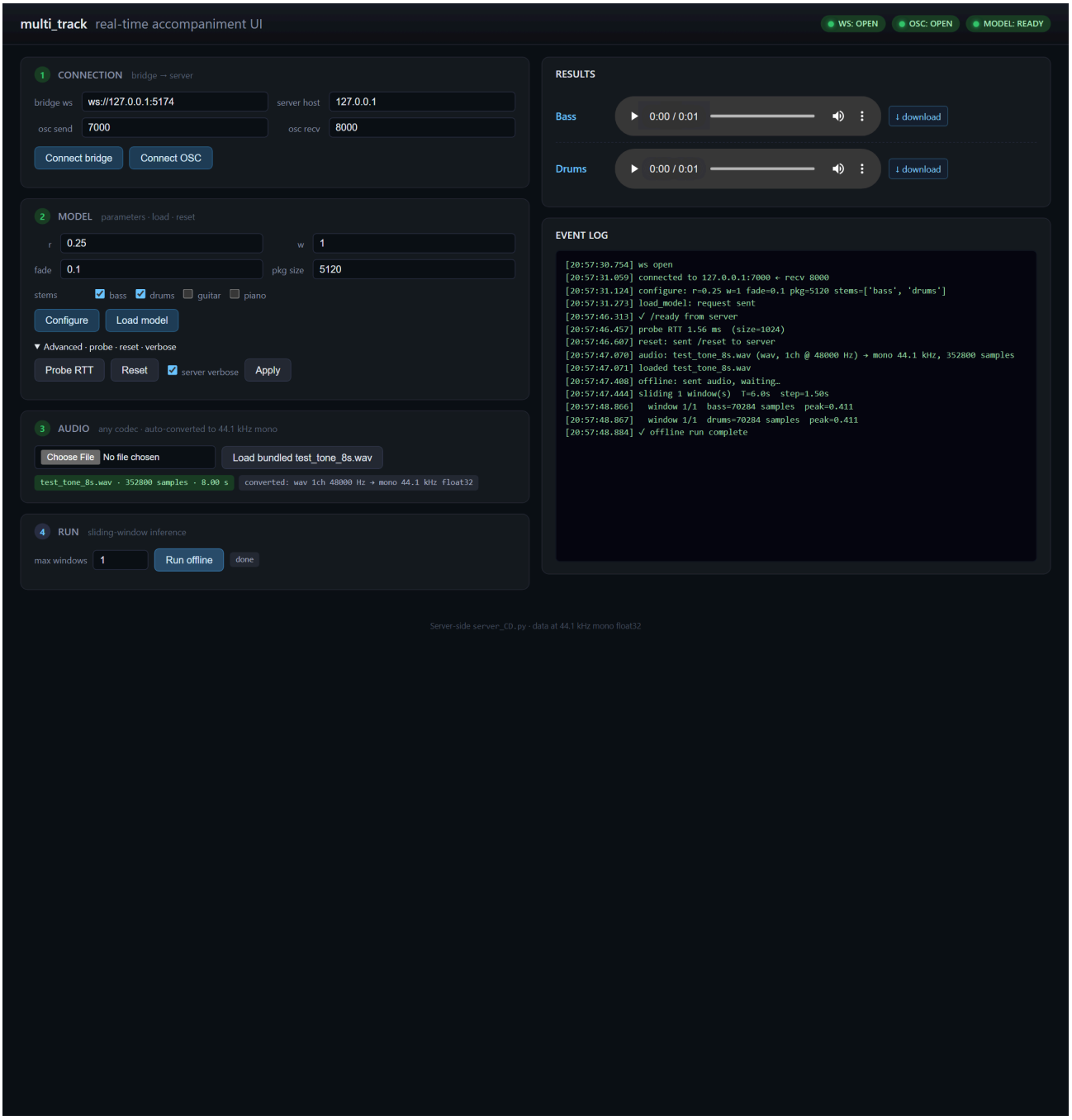
Ghost Note: Migration & Development Action Plan

1. Current State

Ghost Note currently consists of one shared machine-learning backend and multiple clients that drive it over a common OSC protocol. This differs from the original upstream project, which shipped as a single MAX/MSP research demo built directly on top of the Python model.



Operational target state. The current migration is converging on a stack where one shared backend is exercised through visible, testable client surfaces rather than only through the original MAX patch.

The supported entry points are:

- the browser app / web app in `clients/web_ui` for upload-and-run workflows
- the JUCE client in `clients/juce_plugin`, built as both a standalone desktop app and a VST3 plugin
- the Python reference client in `clients/python_ref` for protocol validation and smoke testing
- the legacy MAX/MSP client in `multi_track` for compatibility with the original patch-based workflow

The practical migration direction is clear: the browser app and the JUCE client are the primary forward paths, the Python reference client remains the executable protocol spec, and MAX is retained as a compatibility client until feature parity and operator confidence are high enough to retire it.

2. What The Repository Is Becoming

The repository is becoming a multi-client accompaniment platform rather than a MAX-first paper artifact. Near-term target shape:

- one shared backend for training, checkpoint loading, and OSC inference
- one browser app for low-friction testing, demos, and offline batch runs
- one JUCE-based app/plugin path for DAW and performance workflows
- one reference Python client for diagnostics and protocol-level validation
- MAX maintained only as a legacy bridge until the replacement paths cover the remaining use cases

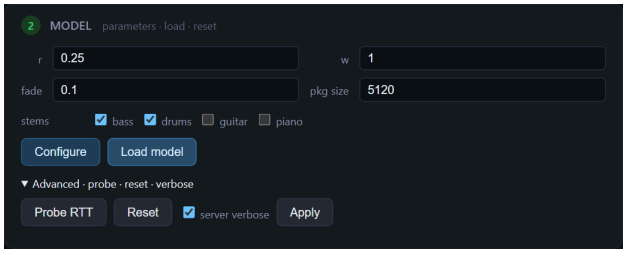
3. Recently Resolved Changes

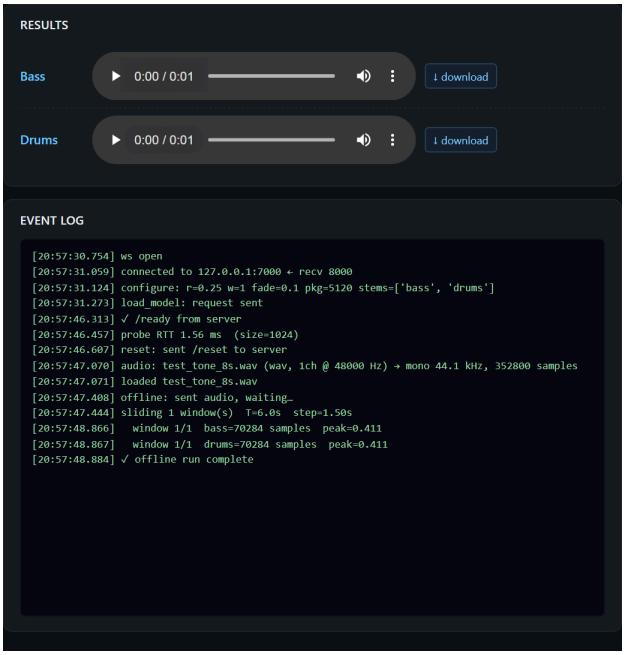
Server Backend (`musical-accompaniment-ldm`)

- **Protocol Upgrade (v1.1):** The `/ready` signal now broadcasts twice (at socket bind and after `/load_model` finishes).
- **Re-connection Reliability:** The `/reset` handler natively flushes per-batch counters (`batch_id` , `auto_chunks_received` , etc.), enabling clients to seamlessly reconnect.
- **Inference Stability:** Refactored audio write-back shape computations in `predict()` to use `flatten_prediction.shape[0]` . This eliminates `tensor size must match` errors during context frame boundary overlaps.
- **Dependency Minimization:** Fully removed `oscpack` reliance via a highly optimized custom `_make_osc_dgram()` implementation utilizing raw `numpy` blocks.
- **Platform Support:** Fully functioning Windows and macOS environment configuration integrated. Added logic for chunk packet spacing as an extra fail-safe.
- **Web and client bring-up:** The recent backend fixes were driven by real client wiring across the browser app, the reference Python client, and the JUCE client rather than only the legacy MAX path.

Frontend clients

- **MAX External (`multi_track`):** Implemented macOS compatibility fixes, patch name changes, and general optimization for demo capabilities.
- **Browser app (`clients/web_ui`):** There is now a browser-driven client with a Python bridge, JSON-over-WebSocket control, bundled test audio, and Playwright smoke coverage for the end-to-end flow.
- **JUCE client (`clients/juce_plugin`):** There is now a C++20 JUCE client that builds as both VST3 and Standalone and speaks the same protocol as the other clients. It currently runs successfully at exactly 44.1 kHz.





Resolved client surface. The browser path already exposes transport setup, parameter control, model load, reset, and probe operations against the shared server.	**Resolved observability surface.** Results, downloadable stems, and a visible protocol timeline now exist in a first-party client instead of only inside MAX.

4. Pending Implementation Tasks (High Priority)

Server Backend (`musical-accompaniment-ldm`)

- **Port v1.1 fixes to `server.py` :** The offline/LDM diffusion backend (`server.py`) has fallen out of sync with `server_CD.py` . It is missing the double `/ready` broadcast logic, chunk counter resets during the `/reset` command, and the tensor shape bug fixes during audio writeback. These modifications must be ported evenly to maintain protocol compliance for clients testing non-realtime high-quality generations.
- **Stabilize client parity assumptions:** Ensure the documented protocol and server behavior remain identical across the browser app, the JUCE client, the Python reference client, and the remaining MAX path. The current repo is close, but the documentation and code can still drift if changes are made in only one client.

JUCE Plugin Refinement

- **Implement Audio Resampling:** Integrate `libsamplerate` or native `juce::Interpolators` into `PluginProcessor.cpp` to decouple the host sample rate from the fixed 44.1 kHz expectation of the LDM framework. Avoid halting/silencing outputs when the host operates at 48k/96k.
- **Persistence Handling:** Implement XML or binary state persistence (`getStateInformation` / `setStateInformation`) for the selected GUI server IP, ports, prediction parameters (`r` , `w` , `fade`), and active generated stems over sessions. Those hooks already exist in the processor but are currently empty.
- **MIDI Assignation Strategy:** Establish MIDI continuous controller (CC) maps allowing musicians to selectively trigger/toggle the four one-hot prediction flags (Bass, Drums, Guitar, Piano) without opening the GUI.
- **Network Resilience:** Finalize timeout routines to facilitate an automatic, graceful UDP reconnect whenever the inference server restarts or networking fails randomly mid-performance.
- **Platform Expansion:** Configure AU (macOS) and AAX build targets in CMake for extended DAW support logic.

Browser App / Web Workflow

- **Clarify scope and keep expanding it deliberately:** The web app is already a real client, not just a demo harness. Decide whether it stays an offline upload-and-run surface or grows into a live capture client. That architectural choice affects the bridge design, test scope, and how aggressively MAX can be retired.

- **Add explicit multi-format documentation and tests:** The web path already normalizes browser-decodable formats to mono 44.1 kHz. The docs and tests should treat that as a supported behavior rather than continuing to document the UI as WAV-only.

Documentation And Onboarding

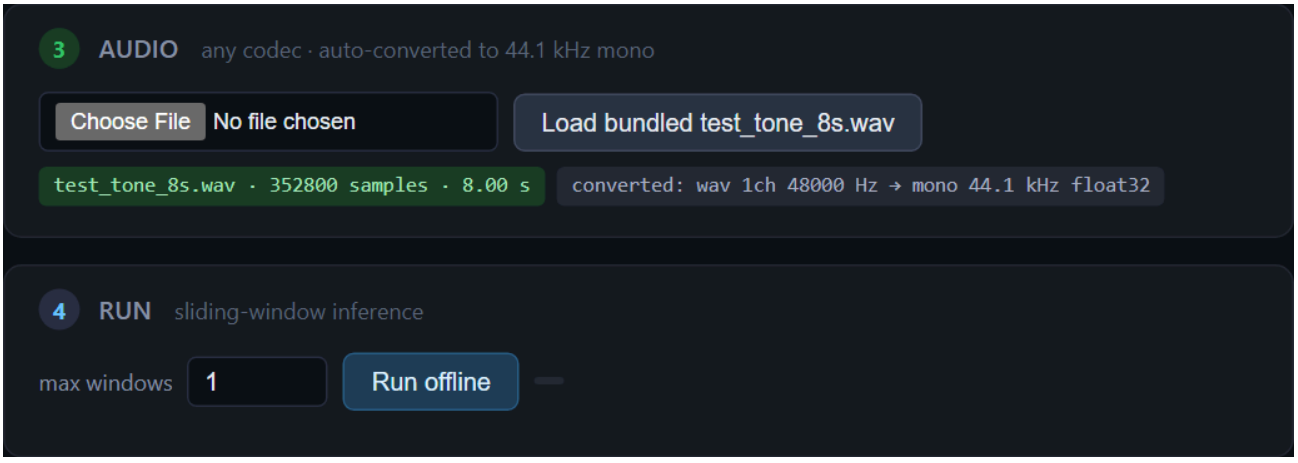
- **Keep the top of every major README aligned:** Root, backend, JUCE, web, and MAX docs should continue to front-load the same message: what the user gets, which path is recommended, and which parts are legacy.
- **Document recommended starting points by audience:** Separate guidance for researchers, DAW users, browser-only evaluators, and legacy MAX operators will reduce confusion and shorten onboarding.

5. Medium / Future Enhancements

- **Deprecating MAX/MSP:** As the JUCE client and browser app reach feature parity and operational stability, formally deprecate `multi_track`. Consolidate deployment efforts around JUCE and web frontends, then move MAX to maintenance-only status.
- **Advanced Parameter Exposing:** Move configuration flags currently embedded within Python definitions to the C++ plugin UI and, where appropriate, the browser app.
- **Transport Lock / Playhead Sync:** Investigate capturing DAW transport states (`play` , `stop` , `loop` via `juce::AudioPlayHead`) to synchronize prediction chunking when the user pauses and restarts rendering.
- **Latency / PDC Adaptation:** Evaluate partial delay compensation reporting for non-realtime offline generation routines or deterministic bouncing, separating them from the direct live loop input.

6. Testing & Validation Strategy

- **Reference Client Upgrade:** Update headless spec routines in `clients/python_ref/test_client.py` and `clients/web_ui` to properly assert and timeout on standard v1.1 protocol rules (assert subsequent `/ready` triggers instead of estimating cold cache bounds).
- **Unit Testing Framework:** Introduce `Catch2` or JUCE `UnitTestRunner` hooks for the DSP bounds and OscBridge class queues.
- **Automation Pipeline Check:** Retain Playwright E2E smoke tests around the JS interface but expand them to explicitly ensure that all four stems return matching context length blocks.
- **Cross-client regression matrix:** Add a small validation matrix covering the browser app, JUCE client, Python reference client, and MAX compatibility path against the same server build and protocol revision.



Validation surface. The browser client already provides a compact end-to-end harness for audio normalization, model execution, artifact download, and visible progress reporting, which makes it a strong anchor for regression coverage.